The Journal of Supercomputing (2023) 79:11427–11451
https://doi.org/10.1007/s11227-023-05114-5

swParaFEM: a highly efficient parallel finite element solver on Sunway many-core architecture

Jingshan Pan[1,2] · Lei Xiao[1] · Min Tian[1] · Tao Liu[1] · Yinglong Wang[1]

Accepted: 11 February 2023 / Published online: 28 February 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract
The simulation of three-dimensional stress and strain is a research hot spot of computational structural mechanics. As the complexity of the project increasing, the size of the matrix generated increases during the simulation. Therefore, a fast and efficient solver is needed. In this paper, we present swParaFEM, a highly efficient parallel finite element solver on Sunway many-core architecture. It is based on preconditioned conjugate gradient iteration algorithm. We launch a master–slave acceleration model to exploit the computational power of Sunway supercomputer. The kernel aggregation optimization scheme is proposed to deal with the problem that threads' frequent creation and destruction waste computing resources. Moreover, we improve the data transfer speed from the slave core to the master core through memory access optimization. Using several optimizations, we achieve a speedup of 10.5× compared to the naive implementation on one compute group of an SW26010-Pro processor and a strong scaling efficiency of 62.8% on 512 compute groups.

Keywords ParaFEM · Sunway architecture · Parallel optimization · Preconditioned conjugate gradient

✉ Min Tian
470939734@qq.com

Jingshan Pan
panjsh@sdas.org

Lei Xiao
qlu_lay_xiao@163.com

Tao Liu
liutao@sdas.org

Yinglong Wang
wangyl@sdas.org

1 Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan, Shandong, China

2 Faculty of Information Science and Engineering, Ocean University of China, Qingdao, China

1 Introduction

Computational structural mechanics [1] has become a powerful tool for large-scale engineering numerical simulation in several scientific fields, such as material properties [2], shipbuilding [3], and earthquake simulation [4]. The simulation of stress and strain plays a significant role in computational structural mechanics. With the increase in simulation accuracy and the requirement of complex physical structure analysis, the scale of matrix equations generated in 3D stress and strain simulation is getting larger and larger. Solving these enormous matrix equations is challengable because of the significant memory overhead. However, the finite element method provides theoretical support for solving large-scale matrix equations by its memory efficiency. Therefore, we adopt the finite element software model to simulate the 3D stress and strain of the structure.

ParaFEM is a general finite element computation software of structural mechanics [5]. Changing the boundary conditions can be widely utilized to solve problems in biology, chemistry, geoscience, engineering, material science, mathematics, and other related fields. ParaFEM is extremely efficient in computing stress and strain, and many scientific research teams have used it to carry out relevant numerical simulations of stress and strain [6]. Due to the limitation of computing power and memory, however, a single computer can't solve large-scale matrix equations.

In recent years, heterogeneous architectures have been widely employed to speed up the simulation process due to their strong floating-point computing power, such as GPUs [7–9] and FPGAs [10–12]. Sunway next-generation supercomputer [13, 14] has strong computing power, a supercomputer independently developed in China, and is powered by the SW26010-Pro many-core processors. Some simulation applications have been optimized on Sunway supercomputer and have an efficient acceleration effect. However, the optimization of ParaFEM software on Sunway supercomputer is still blank.

In this paper, we present a diagonal preconditioned conjugate gradient solver swParaFEM based on PCG(preconditional conjugate gradient) iterative algorithm [15–17]. We ported ParaFEM to the Sunway next-generation supercomputer. After analyzing the performance of ParaFEM, it was optimized in combination with the architecture of SW26010-Pro many-core processor. Our main contributions are as follows:

- *Master–slave acceleration model* We employ MPI+Athread (accelerated thread library) programming mode to realize an efficient master–slave acceleration model for Sunway architecture based on program hot spot analysis.
- *Data transmission optimization* We distribute the subtasks assigned to each master core according to the maximum space limit of the local data memory of slave core and distribute them to each slave core. It improves the utilization of bandwidth and reduces the time consumed due to frequent data transmission.
- *Kernel aggregation optimization scheme* Due to the small amount of computation and frequent calls of the kernel, the thread overhead time is longer

than the computation time. In this paper, we propose an aggregate computing scheme to speed up the stress and strain simulation process of fine structure objects.

- *Memory access optimization* In the process of data transmission in slave core, we employ the DMA (direct memory access) transmission to optimize the speed of accessing main memory and improve the utilization of bandwidth.

The rest of this paper is organized as follows. Section 2 describes the background of 3D stress and strain, preconditioned conjugate gradient algorithm and Sunway many-core architecture, as well as discusses the related work. Section 3 describes the analysis of computing hot spots of naive ParaFEM. Section 4 introduces the optimization scheme of ParaFEM on Sunway supercomputer. Section 5 presents the evaluation results including a comparison experiment of single compute group and multiple compute groups. Section 6 concludes and discusses this paper.

2 Background and related work

In this section, we discuss the related work and introduce preconditioned conjugate gradient algorithm, 3D stress and strain algorithm, and the Sunway many-core architecture. We intend to use [ ] for matrices and { } for vectors.

2.1 Related work

Currently, stress and strain simulation has been applied in many fields of scientific research. Chen et al. [18] used the finite element software to establish the finite element analysis model of T-shaped parts and verified the reliability of the model by using the test data of 10 high-strength steel T-shaped parts and other scholars. Yu et al. [19] proposed to use finite element modeling to compute stress intensity factors. Shen et al. [3] used finite element software to simulate and analyze the mechanical properties of the once cast formwork structure.

Due to the rapid growth of computing requirement, the fem solvers on many-core computers has become a research hot spot. Dai et al. [20] discussed the implementation of practical DG application in three different programming models, as well as some optimization techniques, including grid renumbering and mixed precision to maximize the performance improvements. Yang et al. [21] present a new lattice Boltzmann solver implemented in Taichi programming language, named Taichi-LBM3D. It can be employed on cross-platform shared-memory many-core CPUs or massively parallel GPUs (OpenGL and CUDA). Horikoshi et al. [22] pointed that communication overhead is a critical issue when executing these solvers on large-scale massively parallel supercomputers. The work investigated communication-computation overlapping by asynchronous progress control to various types of preconditioned conjugate gradient methods for parallel finite-element applications. It improves computational performance of Krylov solvers on multicore architectures. Badia et al. [23] proposed three

algorithms that contain a multi-GPU method, and two others are improved versions of algorithms previously developed by some of the authors. In this work, a total of six parallel algorithms implemented using OpenMP and CUDA have been used to carry out the comparison by leveraging the same levels of parallelism on both types of platforms. Experiment result has proven that the work has excellent acceleration for many-core architectures.

ParaFEM enables closer connection between computational structural mechanics and supercomputers. Smith et al. [24] proved the ParaFEM is efficient in computing parallel boundary elements. Then ParaFEM was applied to some scientific computing projects. David et al. [25] proposed a stochastic thermoelastic finite element method with a direct Monte Carlo procedure and used ParaFEM to solve large-scale matrix equations. Tsamos et al. [26] hold that massively parallel computations were required in order to capture damage patterns and evolution meticulously in carbon fibre-reinforced composites (CFRCs). He proposed the feasibility of applying ParaFEM to supercomputers.

Many scientific computing applications have been successfully ported to the supercomputer of Sunway architecture and achieved an excellent acceleration effect [27–32]. Tian et al. [33] proposed a highly efficient parallel sparse direct solver on Sunway many-core architecture based on sparse LU factorization. Xu et al. [34] propose a highly scalable read mapper on the TaihuLight supercomputer optimized for its fourth-generation Sunway many-core architecture. Ye et al. [35] ported OGCMs to next-generation Sunway supercomputer, with excellent program performance.

However, stress and strain simulation of 3D models is still blank on the Sunway architecture. swParaFEM is the first effort to optimize and extend stress and strain simulation on the Sunway architecture.

2.2 Three-dimensional stress and strain

In structural mechanics, the resultant moment of any point in the plane is zero, and the algebraic sum of all forces is zero. Taking the x-axis and y-axis as the projection axis, respectively, combined with Newton's first law, the static equilibrium conditions are shown in Eqs. 1 and 2.

$$\frac{\partial \sigma_x}{\partial x} + \frac{\partial \tau_{xy}}{\partial y} + F_x = 0 \tag{1}$$

$$\frac{\partial \tau_{xy}}{\partial x} + \frac{\partial \sigma_y}{\partial y} + F_y = 0 \tag{2}$$

where σ_x, σ_y and τ_{xy} are the only nonzero stress components and F_x, F_y are the components of x-axis and y-axis, respectively. However, the two differential equations include three unknowns, which cannot be solved. Therefore, the problem of determining the stress component is statically indeterminate, and the deformation must be considered to solve the problem.

$$\begin{Bmatrix} \sigma_x \\ \sigma_y \\ \tau_{xy} \end{Bmatrix} = \frac{E}{1-v^2} \begin{bmatrix} 1 & v & 0 \\ v & 1 & 0 \\ 0 & 0 & \frac{1-v}{2} \end{bmatrix} \begin{Bmatrix} \epsilon_x \\ \epsilon_y \\ \gamma_{xy} \end{Bmatrix} = [A] \begin{Bmatrix} \epsilon_x \\ \epsilon_y \\ \gamma_{xy} \end{Bmatrix} \tag{3}$$

where E is Young's modulus, v is Poisson's ratio and x, y and γ_{xy} are the independent small strain components. For the motion of any object, the equation of motion can be written by momentum theorem or Newton's second law. The differential equation of motion of displacement caused by strain, also known as Cauchy equation, can be expressed in the form of matrix, as shown in Eq. 4.

$$\begin{Bmatrix} \epsilon_x \\ \epsilon_y \\ \gamma_{xy} \end{Bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x} & 0 \\ 0 & \frac{\partial}{\partial y} \\ \frac{\partial}{\partial y} & \frac{\partial}{\partial x} \end{bmatrix} \begin{Bmatrix} u \\ v \end{Bmatrix} = [D] \begin{Bmatrix} u \\ v \end{Bmatrix} \tag{4}$$

where u and v are the components of displacement in the x-and y-directions. When the above equation is extended to three-dimensional displacement components u, v and w, the three simultaneous partial differential equations are equivalent to the following results. Discretisation proceeds as usual, and again the familiar element stiffness properties are derived, as shown in Eq. 5.

$$[k_m] = \iiint ([A][S])^T [D]([A][S]) \mathrm{d}x\mathrm{d}y\mathrm{d}z \tag{5}$$

where the $[k_m]$ is the element stiffness matrix. The [S] is the shape function stiffness matrix. With the above information, we are able to deduce the full strain–displacement relations and abstract it as a matrix for computation.

In this paper, we solve the large-scale matrix equation based on the above formula and PCG algorithm.

2.3 Preconditioned conjugate gradient

Conjugate gradient method is a method between gradient descent method and Newton method. It overcomes the disadvantage of slow convergence of gradient descent method and avoids storing and calculating the second-order derivative information required by Newton method. Since one direction is optimized to the minimum every time, the subsequent optimization process will no longer affect the minimum value in the previous optimization direction. The method is often used to solve large systems of linear equations.

For the static equilibrium problem, using the Gaussian elimination method, all the element $[k_m]$ matrices and f vectors are combined into a system of linear homotopy equations of "global" form. If this space is not available, out-of-core techniques cause a serious deterioration in analysis speeds. But we can use the solution strategy of conjugate gradient iteration instead of Gaussian elimination. The conjugate gradient method is used to solve the linear equations as shown in Algorithm 1.

Algorithm 1 PCG algorithm

1: $\{P_0\} = \{R_0\} = \{F\} - [K_m]\{U\}_0$
2: Repeat
3: $\{Q\}_k = [K_m]P_k$
4: $\alpha_k = \dfrac{R_k^T R_k}{p_k^T Q_k}$
5: $\{U\}_{k+1} = \{U\}_k + \alpha_k\{P\}_k$
6: $\{R\}_{k+1} = \{R\}_k - \alpha_k\{Q\}_k$
7: $\beta_k = \dfrac{\{R\}_{k+1}^T\{R\}_{k+1}}{\{R\}_k^T\{R\}_k}$
8: $\{P\}_{k+1} = \{R\}_{k+1} + \beta_k\{P\}_k$

where $\{R_0\}$ is the residual or error for a first trial $\{U\}_0$. Until the difference between $\{U\}_{k+1}$ and $\{U\}_k$ is sufficiently small, as determined by a convergence criterion. In the above, $\{Q\}$, $\{P\}$ and $\{R\}$ are vectors of length equal to the number of equations to be solved (neq in programming terminology), while α and β are scalars. Vitally, however, if $[K_m]$ is a system stiffness matrix, and all that is required is the product $[K_m]\{P\}$ where $\{P\}$ is a known vector, this product can be carried out element-by-element without ever assembling $[K_m]$ at all, as shown in Eq. 6.

$$\{Q\} = \sum_{i=1}^{\text{nels}} [K_m]_i\{P\}_i \tag{6}$$

where nels is the number of elements, $[K_m]_i$ is the element stiffness matrix of the ith element and $\{P\}_i$ the appropriate part of $\{P\}$. The storage required by such an algorithm, would be an order of magnitude less, and would grow linearly with the increase in number of elements or equations rather than as the square. In practice, preconditioning can be used to accelerate convergence of some iterative processes for solving the element-by-element version, as shown in Eq. 7.

$$[P][K_m]\{U\} = [P]\{F\} \tag{7}$$

With excessive computational effort, $[P]$ could be calculated as the inverse of $[K_m]$ and the solution obtained in one step, as shown in Eq. 8.

$$\{U\} = [K_m]^{-1}\{F\} \tag{8}$$

In practice it turns out that relatively crude approximations to $[K_m]^{-1}$ can be used to construct $[P]$ and hence be used in the iteration process. For example, diagonal preconditioning uses the inverse of the diagonal terms of $[K_m]$ as a vector $\{P\}$.

ParaFEM mainly uses the PCG algorithm to go for simulation experiments. The major work of this paper will be on the optimization of the PCG algorithm.

2.4 SW26010-Pro processor

The Sunway next-generation system is based on the SW26010-Pro many-core processor. Sunway's next-generation multicore processor (SW26010-Pro) consists of six compute groups. Each compute group contains a management processing element (MPE) and an array of 8*8 compute processing elements (CPE), as shown in Fig 1. These components are connected through a ring network. The main functions of the MPE are computation, control and communication. The CPEs are mainly used in computation. The data transmission between CPEs and MPE is interconnected through the network in the array.

MPE uses a separate sw64 instruction and sets with 32KB L1 instruction cache, 32KB L1 data cache and 512KB L2 cache. CPE supports double-precision, single-precision, and half-precision floating-point computation. Each CPE has its own instruction cache and data storage. The data storage can be configured as a fully user-controlled local data memory (LDM). In fact, peak bandwidth is difficult to achieve due to alignment requirements and read/write interleaving. Another bottleneck is that the 256KB LDM is not enough to storage giant data. Since these inherent limitations of the SW26010-Pro architecture, the computation tasks should be divided into smaller blocks.

3 Hot spot analysis

In this section, we analyze the hotsopt of ParaFEM software.

The naive ParaFEM parallelism strategy is process-level parallelism, and there is no thread-level parallelism optimized for the structure of Sunway's CPE. In order to develop an efficient and reliable parallel strategy suitable for the Sunway

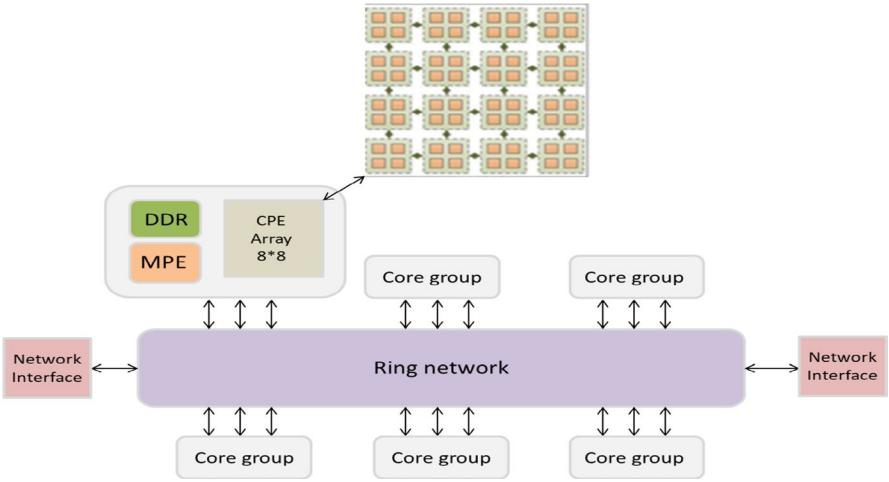


Fig. 1 Sunway many-core processor architecture

Table 1 ParaFEM hotsopt analysis

Module	Time (s)	Percentage
Input module	114.82	27.85
Assembly module	1.56	0.37
Solve module	241.37	58.54
Other module	54.54	13.24

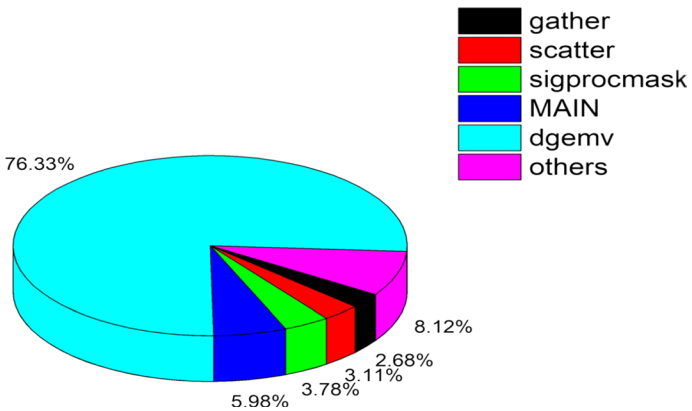


Fig. 2 Proportion time of each main function

supercomputer architecture, we analyzed the performance of the ParaFEM program using the SWPROF tool.

ParaFEM consists of four modules that are the read data module, the assembly module, the solver module and the other module. The model II in Table 5.2 was selected for the hot spot analysis experiments. We execute the ParaFEM program with 512 processes. Table 1 shows the running time and the percentage of time spent for each module. The solve module is the major computation module of ParaFEM and reaches 54.84% of the total running time. The input module reaches 27.85% of the total running time. We plan to change from single-process reads to distributed reads in future work. The other module reaches 13.24% of the total running time. Since ParaFEM does not take the assembly global matrix, the assembly module takes 0.37% of the total running time. Therefore, we do not optimize the assembly module. Because the solve module takes the longest time, we choose to optimize the solve module. However, the solve module calls a number of third-party library functions, and it makes optimization difficult. To find the specific algorithm to optimize, we have inserted a timestamp inside the solve module to count the total time of the function calls, as shown in Fig. 2.

The dgemv kernel reaches 76.33% of the total running time in solve module. This kernel is mainly used to perform the operation of multiplying vectors with matrices. The ParaFEM mainly uses the degmv kernel for PCG iterative update operations. Therefore, we improve the performance of ParaFEM by optimizing the PCG algorithm and the dgemv kernel.

In addition, we also derived the program flow of ParaFEM by hot spot analysis, as shown in Fig. 3. The process of solving the equations can be performed by the pcg technique without assembling the element matrix into a global (stiffness) matrix. We optimize ParaFEM in conjunction with the hot spot analysis result and further improve the computational efficiency of the ParaFEM program.

The program has a high number of dgemv calls. However, the dgemv kernel is not computationally intensive. If a thread is created every time the dgemv kernel is called, the overhead of the thread is greater than the overhead of the computation, making the thread overhead time longer than the computation time. We need to find a strategy for aggregating these same kernels to expand the amount of computation when a thread is created. The reasonable data transmission scheme should be designed to improved bandwidth utilization correspondingly. In this paper, the optimization of ParaFEM is carried out along the hot spot.

4 ParaFEM optimization on Sunway architecture

In this section, we introduce the master–slave acceleration model, data transmission optimization, kernel aggregation and memory access optimization on Sunway architecture.

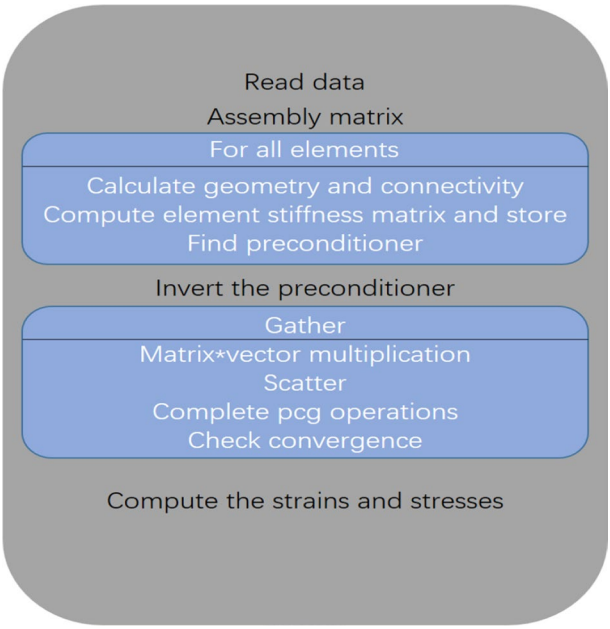


Fig. 3 The program flow of ParaFEM

4.1 Master–slave acceleration model

Based on the hot spot analysis, we have identified an efficient and reliable hierarchical parallel scheme of ParaFEM for Sunway architecture. The first level is the process-level parallelism by MPI. swParaFEM assigns the grid to each MPE based on the total number of grids and the number of nodes running the program. Then, when the MPE receives the corresponding number of grids from MPI, it starts to read the data and allocates enough memory to the size of the data. Each grid computes the geometry and connectivity. It also computes and stores the mesh stiffness matrix and searches the preconditions. Due to the small amount of computation, we have computed these processes in MPE. After reversing the obtained preconditions, the PCG iterative process was performed. The PCG iteration is the computation-intensive part of the program. Therefore, we assign the PCG iteration part from MPE to CPE for computation. In CPE, the computation part is performed with the help of Athread accelerated thread library. When the CPE computation is completed, the computation results are transferred to the main memory. The master–slave parallel mode is shown in Fig. 4.

The second level is the thread-level parallelism among CPEs. In this process, MPE is mainly used for process control and memory allocation. The CPE directly accesses to the main memory address through DMA technique in data transmission. The CPE stores the data in its LDM and then performs the computation. After the computation, the CPE transfers the updated data back to the main memory through DMA technique. The computation pattern between MPE and CPE is shown in Fig. 5.

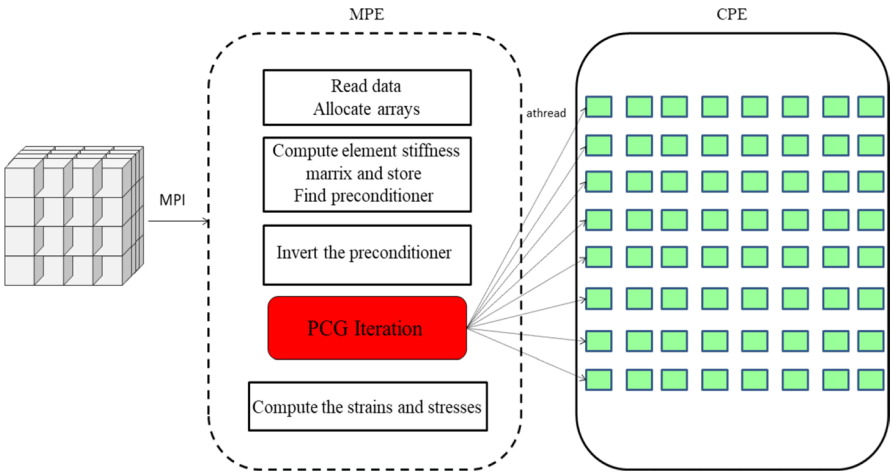


Fig. 4 Master–slave acceleration model

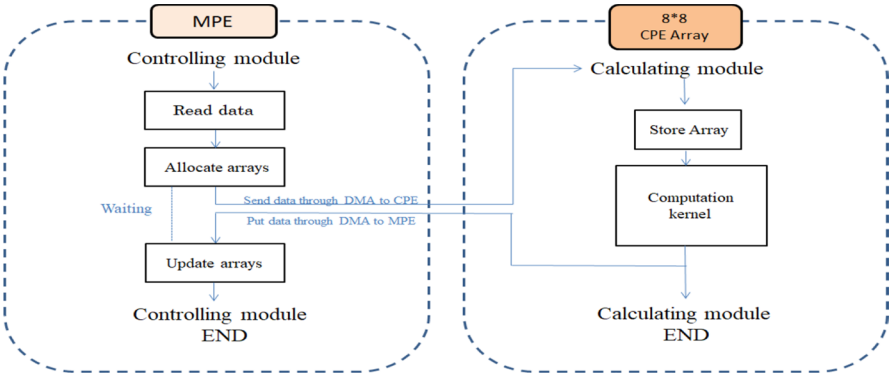


Fig. 5 Computing mode between MPE and CPE

4.2 Data transmission optimization

In this paper, we divide the computation tasks based on the computation grid. The computation grid tasks assigned to each MPE. Then the grid computation tasks are divided equally to each CPE in the second-level parallel computation. Since each compute group has 64 CPEs, we divide the grid equally to each CPE for load balancing. Among them, the intensive computing part employs MPI to slice the grid along the x-axis and assign it to each MPE. Since grids assigned to each MPE are the same, the DMA requests from CPE to the main memory address needs to be considered. The detailed data partitioning is shown in Fig. 6.

In order to improve the computational efficiency of each grid in CPE, we analyzed theoretically the storage efficiency of dynamic arrays and static arrays. Dynamic arrays are created on the heap. For the heap, frequent allocation and release of heap space of different sizes are bound to cause discontinuity of memory space, resulting in a large number of fragments and reduced program efficiency. For the stack, this problem does not exist. For static arrays, they are created on the stack, and the stack memory allocation operation is built into the instruction set of the processor. Its operation efficiency is generally very high. In order to improve the speed of accessing memory and ensure continuous memory reading, a one-dimensional array can be used for a two-dimensional array. Therefore, this paper adopts the form of a one-dimensional static array to store the data transmitted from the main memory to the CPE.

Each grid computation depends on several arrays. However the limitation of the kernel LDM space, this paper carries out different data transmission according to the specific size of the array. For large arrays, this paper transfers them circularly in the computation process. For some arrays with low dimensions, we adopt one-time transmission before computing from the core to increase the reusability of data. Data transmission modes are shown in Algorithm 2.

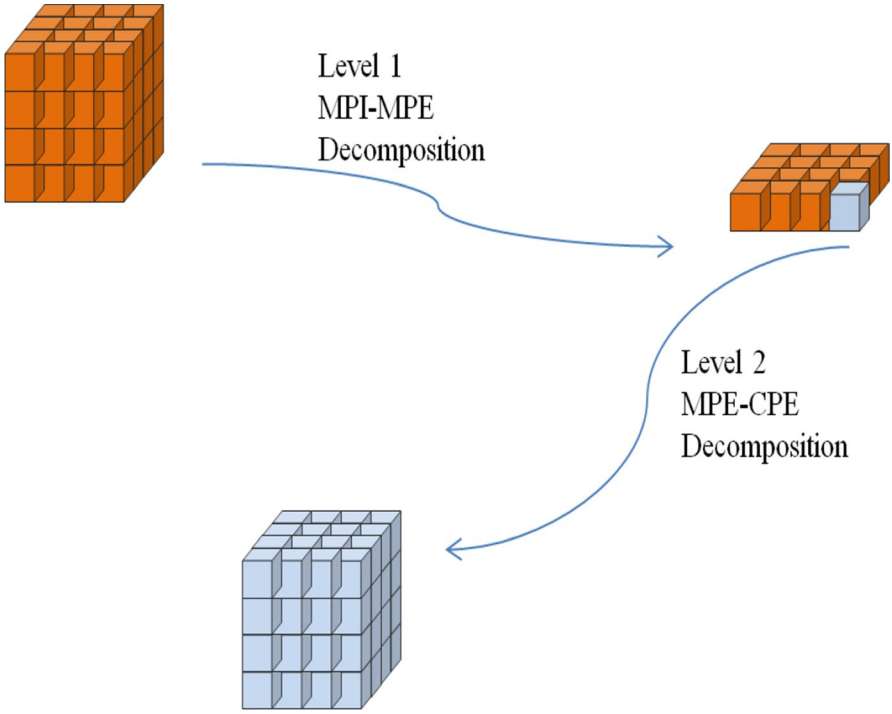


Fig. 6 Two-level data partitioning

Algorithm 2 PCG data blocking algorithm

```
Input: matrix A , matrix B , ... , matrix N
Init martix in MPE
Read matrix in MPE
Allocate matrix in MPE
/*Traverse all of matrix */
for matrix K = 1 to N do
/*Judge the amount of matrix and LDM space*/
    if sizeOf(matrix K) > LDM then
        Create loop in CPE
        Cut matrxi K to LDM in loop
    else
        if sizeOf(matrix K) < Available LDM then
            Send the matrix K to LDM
        else Cut the matrix K to LDM
        end if
    end if
end for
```

Based on the block transmission method, we improve the data reuse in local memory and the utilization of LDM. The data transmission of fewer arrays outside the cycle can fully utilize local memory, reduce the amount of data in DMA transmission, and improve computing performance.

4.3 Kernel aggregation optimization scheme

According to the above kernel execution process, MPE traverses each grid and performs matrix–vector multiplication using the degmv kernel. Since each MPE is allocated to several grids, the kernel relies on more extensive data. Therefore, the 256KB LDM limited the efficiency of the kernel. When the number of loops increases, the program creates and destroys threads frequently, which leads to a decrease in acceleration or even negative values. The dgemv kernel's hot spot analysis after kernel optimization is shown in Fig. 7.

The athread_spawn represents the time related to thread overhead, while the athread_join represents the computing time in CPE. It can be seen that the time spent on creating and destroying threads exceeds the time spent on computation.

In this paper, we propose a kernel aggregation optimization scheme, which is suitable for loop optimization with small amount of computation and large number of cycles. To solve the problem, we use a split-loop format for optimization. Due to the computational power of CPE is much greater than the computational power required by this kernel, it is necessary to properly execute multiple identical kernels at the same time in CPE. After the loop is split, the loop slices are assigned to each CPE. The kernel is no longer called through MPE but is implemented inside each

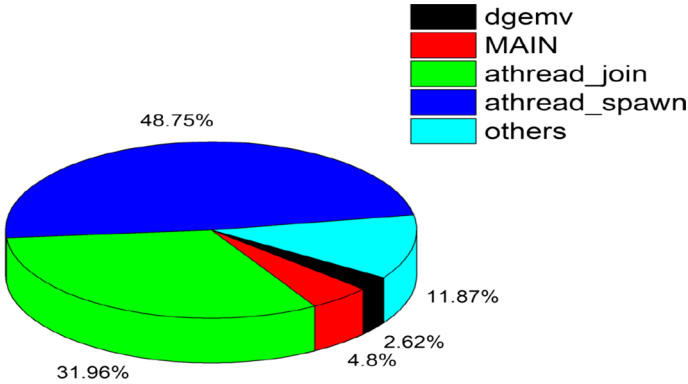


Fig. 7 Proportion time of each main function after master–slave acceleration optimization

CPE. This optimization method can increase the computation amount of CPE, compress the time of thread create and thread destruction, and make further use of the powerful computing power of CPE. Our strategy is more suitable for kernels with smaller computation and a larger number of loops. Taking the optimized kernel as an example, the detailed algorithm is shown in Algorithm 3.

Algorithm 3 Kernel Aggregation Optimization algorithm

```
Input: LoopFrag A , LoopFrag B , ... , LoopFrag N
/*Traverse the circular fragment*/
for LoopFrag K = 1 to N do
/*Judge the amount of thread overhead and computation*/
    if Loop dimension * Computation <  CPE thread overhead  then
        Up one dimension to cut the Loop K
        Send the Loop K to the CPE
        Create thread to accelerate Loop K
    else
        if sizeOf(matrix K) >  Available LDM then
            Create the CPE kernel to accelerate Loop
        else Cut the matrix K to LDM
        end if
    end if
end for
```

The time consumption caused by frequent thread creation is unacceptable. Therefore, this paper adopts the aggregated dgemv kernel to avoid the problem of the thread overhead. Each CPE only needs to create a thread once to call the dgemv kernel many times to compute the matrix–vector multiplication. The thread overhead time is greatly reduced, the computing time of CPE is greatly increased, and the computing performance is further improved. The optimized result is shown in Fig. 8.

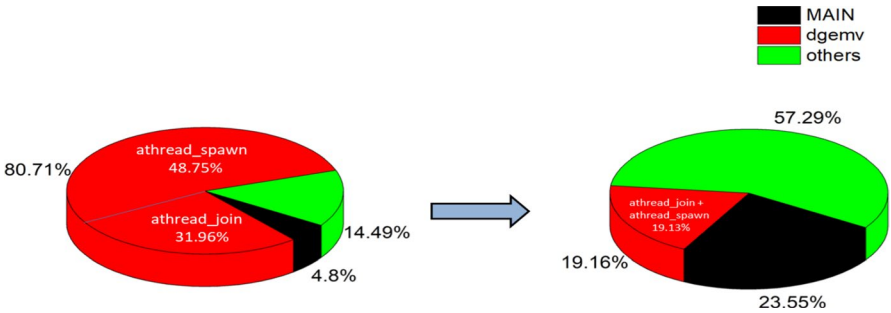


Fig. 8 The performance analysis after kernel optimization

Generally, we employ the ratio of computing time and thread overhead time to measure the computing efficiency of a program. The ratio is proportional to the efficiency of the thread computation. Table 1 shows that the time of athread_spawn decreased from 48.75 to 0.03%. It shows that the time of thread creation and destruction is greatly reduced, the actual computation time is longer, and the computation utilization rate is higher. Therefore, the results show that the kernel aggregation scheme has excellent applicability for reducing the thread overhead time and improving the computing time.

4.4 Memory access optimization for CPE

Under the architecture of SW26010-Pro processor, the CPE has a 256KB local memory (LDM), which is organized in the form of scratch pad memory (SPM) and controlled by software. It belongs to a programmable memory. By default, the CPE can directly access the main memory by load/store instructions. The form of direct access to main memory is called discrete access to main memory, which can flexibly and efficiently access any main memory address. However, discrete access to main memory requires about 278 clock cycles, which is also a relatively high overhead with the growth of data scale. Therefore, we propose to use DMA (direct memory access) transmission to access the main memory in batch. Each batch access to the main memory only needs about 4 cycles, which is nearly 70 times faster than that of discrete access to the main memory, as shown in Fig. 9.

When data are aligned with memory particles, the operation can be completed only by one DMA transmission. However, if the data are not aligned, two DMA transmissions are required, as shown in Fig. 10. The optimization of CPE memory access by DMA transmission can speed up the data transmission between CPE and main memory. It also improves bandwidth utilization in the case of data alignment.

In this paper, DMA transmission is employed as the transmission method to accelerate computing, which can improve the speed of data access. In addition, we have improved the efficiency of DMA transfers by rationalizing the data blocks. The utilization of memory bandwidth is further improved. It also improves performance of the program.

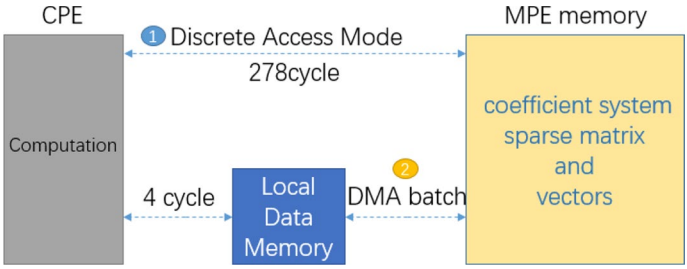


Fig. 9 DMA technique was used transfer data from MPE to LDM in batch

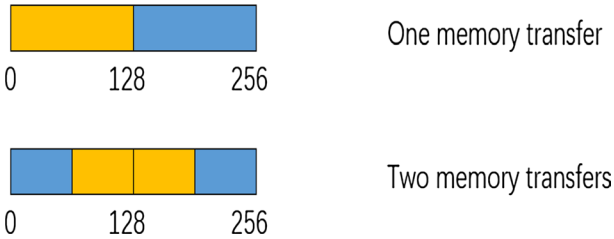


Fig. 10 The characteristics of DMA-aligned data transfer

Table 2 Environmental parameters

Parameter	Value
CPU	SW26010-Pro
Number of CGs	6
Number of processors of one CG	65 (1 MPE + 64 CPE)
Memory size	96GB
Scratch pad memory	256KB
Precision support	Double-precision
	Single-precision
	Half-precision

5 Evaluation

5.1 Setup

In this section, the major computing part of swParaFEM is evaluated and discussed the diverse parallel acceleration schemes on the Sunway next-generation supercomputer. The 'process number' means number of compute groups that consist of a MPE and 64 CPEs. The specific experimental environment is shown in Table 2.

The evaluating version of ParaFEM is 5.0.3. In this paper, PCG finite element solver is selected to evaluate the optimization effect of different methods. The evaluating example is a hexahedral elastomer, the lower boundary of which is a stationary

surface, and the upper boundary is entirely free. The front and left sides are symmetrical planes. The back and right sides are external boundaries. The block has layered properties, with the upper and lower halves assigned Young's moduli values of 100 and 50, respectively. Poisson's ratio is fixed at 0.3. The grid is a 100*100*100 three-dimensional model generated by the grid generation tool p12meshgen of ParaFEM. This example is used for the strong scaling analysis and the overall performance analysis of ParaFEM program. In addition, a 20*20*20 three-dimensional model is also generated for the related test of single-core group. Details of the model are shown in Table 3 and Fig. 11.

5.2 Parallel optimization evaluate and analysis

In order to evaluate the performance of the optimization strategy on SW26010-Pro heterogeneous many-core processors, this section evaluates the parallelism effect on MPE and CPE, respectively. We also evaluate the fine-grained optimization method on the master–slave hybrid accelerated. We employ the 1 MPE and 64 CPEs to ran the model I. The evaluation results are shown in Table 4.

Table 3 Model parameter

Parameter	Model I	Model II	Meaning
grid	20*20*20	100*100*100	Total number of elements
nn	35,721	4,090,601	Total Number of nodes in the mesh
nr	6081	150,401	Nodes with constrained degrees of freedom
equations	98,360	12,059,800	Number of equations

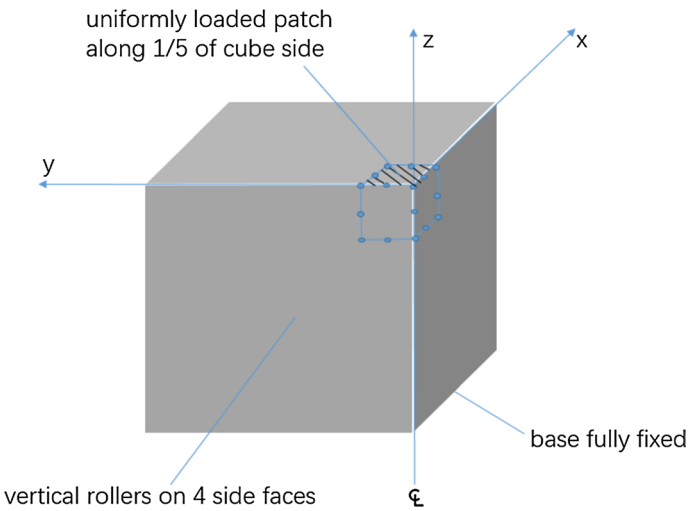


Fig. 11 The elastic solid model structure

Table 4 Acceleration effect of the major computing segment

Version	Process number	Time/s
Serial program	1 MPE	203.11
MPI program	4 MPE	53.15
MPI+Athread	1 MPE + 64 CPE	19.33

As shown in Fig. 12, after using several optimization strategies to optimize the ParaFEM, PCG iteration of swParaFEM achieves 10.5× speedup for naive implement. For the master–slave optimization of dgemv kernel, there are some speed improvements. The major computing section accounts for about 76% of the serial program. However, the unoptimized dgemv kernel has a poor acceleration effect. In one compute group test, the dgemv kernel's efficiency is not as great as expected. However, when the matrix dimension is not large, the utilization rate of CPE's LDM is low. The bottleneck of frequent thread creation makes the thread overhead time occupy 48.75% of the program running time. In contrast, the major computing time is only 31.96%. Because the time of thread overhead is expensive, we employ kernel aggregation optimization scheme to deal with it.

The xMath is not one of the iterative versions produced in this paper. xMath is a many-core BLAS library that is a high-performance computing interface with high scalability. Compared with the xMath, PCG iteration of swParaFEM achieves 6.67× speedup. The reason is that through grids division, we make the amount of computation on each CPE as large as possible and reduce the threads overhead.

The performance of optimization strategies for swParaFEM are shown in Fig. 12. The optimization effect of the program is related to multiple factors. These

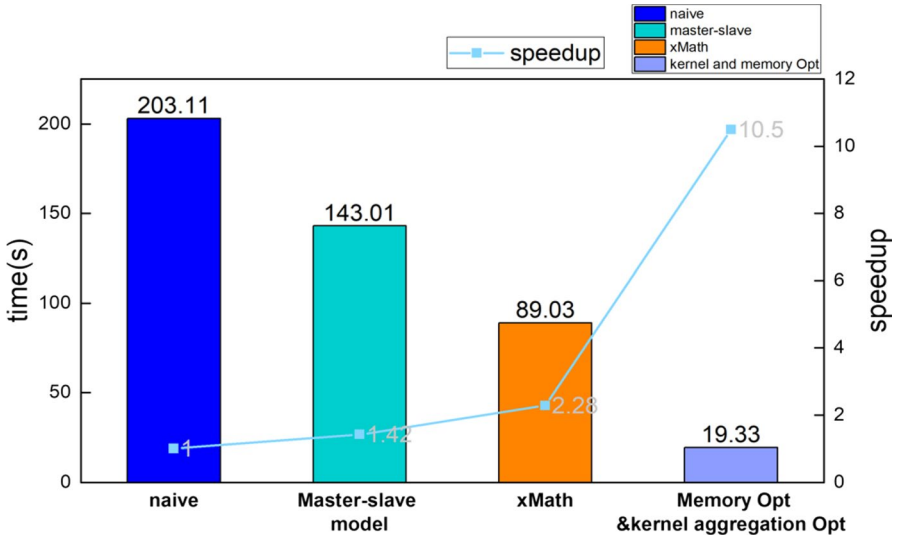


Fig. 12 Runtime after incrementally applying each optimization

optimization strategy is applied, PCG iteration of swParaFEM achieves 10.5× speedup. This paper replaces the original single-level parallelism with two-level parallelism by master–slave optimization. The two-level parallel algorithm involves the data transfer between CPE and MPE. Therefore, we propose the transfer algorithm to solve the data transfer problem. We propose the kernel aggregate scheme to solve the problem that the kernel computation time is less than the thread overhead time. Finally, we optimize the memory access problem between master and slave kernels by using DMA technique. In summary, these optimization strategies are closely fitted and effective for improving the performance of ParaFEM programs.

After the experiment, we derived the post-processed image from paraView as shown in Fig. 13.

5.3 Scalability

In this section, we carry out a large-scale strong scaling analysis on the major segment of the above-optimized version of swParaFEM. We adopt the model II for evaluating. During the analysis, the number of meshes of the model remain unchanged, and the number of processors used increase in proportion.

As is shown in Table 5 and Fig. 14, the results show strong scaling test for swParaFEM on Sunway next-generation supercomputer with model II. Due to the limited memory of MPE, the model II can't run on a compute group. We have to start the analysis with 4 compute groups. The acceleration of the parallel accelerated solution program increases approximately linearly relative to the serial program. When the computational force scale is further expanded, the total number of computational iteration cycles in the program increase exponentially, and the load assigned to each CPE increase accordingly. It can be seen from the analysis results that under the analysis scale of 100*100*100, the acceleration effect of swParaFEM

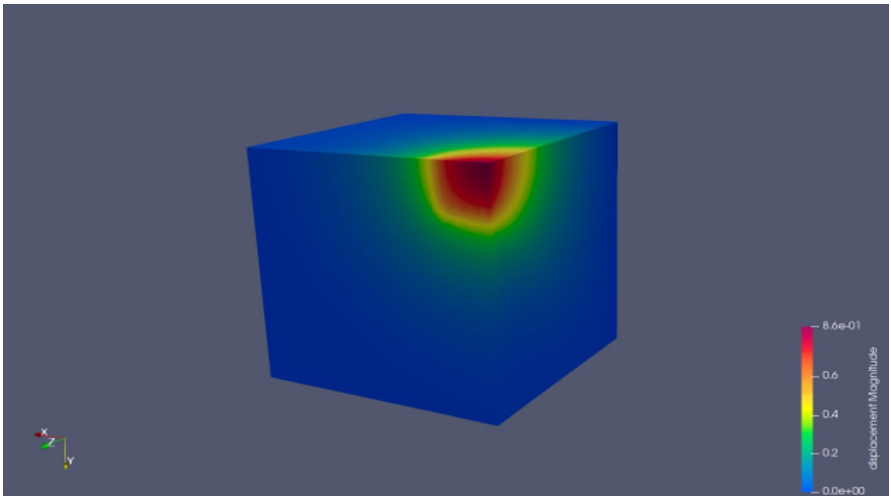
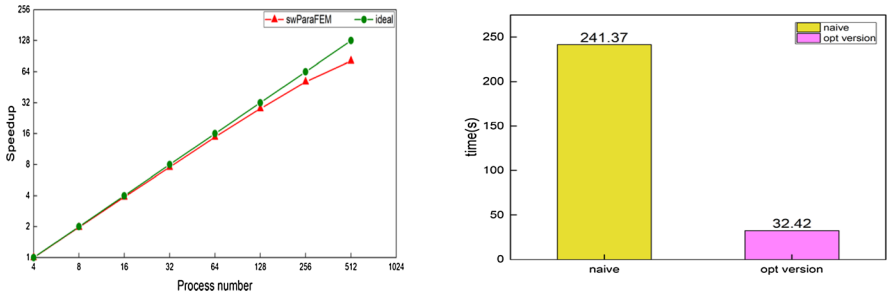


Fig. 13 ParaView post-processing result

Table 5 Evaluation of swParaFEM scalability

Processes	4	8	16	32	64	128	256	512
Time (s)	2606.23	1320.16	671.59	345.67	176.41	93.61	51.41	32.42
Efficiency	100%	98.71%	97.02%	94.25%	92.34%	87.00%	79.21%	62.80%



(a) Strong scaling analysis (b) Performance comparison with 512 processes

Fig. 14 The performance of paraFEM's strong scalability

Table 6 Evaluation of assembly time and solving time

Processes	4	8	16	32	64	128	256	512
Assembly time (s)	163.19	80.75	42.18	20.66	10.37	5.43	2.9	1.56
Solving time (s)	2606.23	1320.16	671.59	345.67	176.41	93.61	51.41	32.42

program with 512 compute groups is effective. Compared with the serial program, the acceleration ratio can always maintain a linear growth, and the efficiency of parallel program to the compute group will hardly be reduced, indicating that the optimized solution program has excellent scalability on Sunway next-generation supercomputer platform.

In order to make the experimental data more transparent and convincing, we list the assembly time and the solving time separately, as shown in Table 6.

5.4 Performance of swParaFEM

In this section, in order to ensure the overall performance superiority of swParaFEM software, we evaluate and analyze the overall performance of swParaFEM.

Table 7 shows the speedup evaluate results of swParaFEM on Sunway next-generation supercomputer with model II. The swParaFEM achieves 9.78× speedup with 4 compute groups, as shown in Table 5, achieves 151.71× speedup with 1000 compute groups. The process number means compute groups.

Table 7 swParaFEM evaluate

Version	Process number	Time/s	Speedup
MPE	4	29,155.26	1
MPE + CPE	4	2980.56	9.78
MPE + CPE	1000	192.17	151.71

As is shown in Fig. 15, the performance of the first few points was fairly significant, and the acceleration ratio basically increased exponentially. However,due to the large scale of the model, MPI communication will be affected with the increase in the number of processes. It can be found that the communication time of MPI is basically unchanged, and the computation time of the major segment is even lower than that of MPI.

Next, we will study and discuss the communication problem deeply. And we will further improve the parallel efficiency of swParaFEM.

6 Conclusion and discussion

Conclusion In this paper, we propose a diagonal preconditioned conjugate gradient solver swParaFEM based on PCG iterative algorithm. swParaFEM uses an efficient memory iterative method to solve the system of matrix equations. Due to the advantages of PCG algorithm, it is not necessary to put the whole matrix into memory, which makes it possible to solve the super large-scale matrix equation for SW26010-Pro processor. We adopt optimization methods such as CPE memory

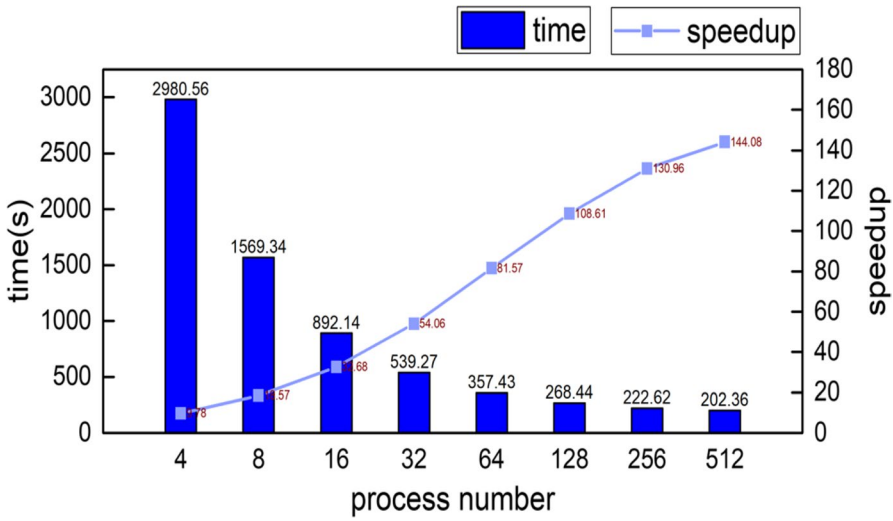


Fig. 15 Strong scaling analysis of swParaFEM

access optimization, efficient multilevel parallel strategy, parallel blocking strategy for PCG algorithm, kernel aggregation optimization scheme. Using several optimizations, we achieve a speedup of 10.5× compared to the naive implementation on one compute group of an SW26010-Pro processor and a strong scaling efficiency of 62.8% on 512 compute groups.

Discussion Zhu et al. [36] optimize the HPCG benchmark based on Sunway supercomputing. The work proposes a two-level blocking scheme exposing sufficient parallelism symmetric Ghosseidel cores while maintaining fast convergence speed, fine-grained core fusion techniques to alleviate bandwidth load on local storage, and a low-overhead thread collaboration approach to efficiently move data between threads and hide costly operations with data transfer. In verifying the scalability of the program, we used a strong scalability test, which takes a weak scalability test. This may be caused by the different definition of the problem. And it scales the HPCG benchmark to 42 million heterogeneous cores while this paper does not need to scale to that many cores for computation due to the limitation of the problem size. Overall this work is a guideline in the direction of Shineway's optimization work, and our next step is to learn from it to further increase the scalability of our program.

For the choice of diagonal preconditioner, there is still a lot of room for optimization in this paper. Badia et al. [37] propose the multilevel domain decomposition, it present a fully distributed, communicator-aware, recursive, and interlevel-overlapped message-passing implementation of the multilevel balancing domain decomposition by constraints (MLBDDC) preconditioner. The implementation highly relies on subcommunicators in order to achieve the desired effect of coarse-grain overlapping of computation and communication, and communication and communication among levels in the hierarchy. Using it as a diagonal preconditioner improves the scalability of the program and further accelerates the solution efficiency. Fabien et al. [38] use GMG method achieves ideal convergence rates of 0.2 or less on the Intel's second-generation Xeon Phi (Knights Landing) many-core processor, for high polynomial orders. A matrix-free (assembly-free) technique is exploited to save considerable memory usage and increase arithmetic intensity. Its work with the Intel architecture is excellent. It makes us consider whether we can port its algorithms under Intel architecture to be optimized again in Sunway architecture. This would not only enrich the algorithm ecology of Sunway, but also improve the scalability and computational speed of the program.

Acknowledgements This work was supported by National Natural Science Foundation of China under Grant 62002186, ShandongYouth Innovation Talent Introduction and Education Plan (Parallel Computing Industrial Software Innovation Team Based on Chinese Supercomputer), and the Qingdao National Laboratory for Marine Science and Technology under Grant No. 2018ASKJ01.

Author contributions LX and JP wrote the main manuscript text and others prepared figures 1–10. All authors reviewed the manuscript.

Funding This work was supported by National Natural Science Foundation of China under Grant 62002186, the Qingdao National Laboratory for Marine Science and Technology No.2018ASKJ01, and ShandongYouth Innovation Talent Introduction and Education Plan (Parallel Computing Industrial Software Innovation Team Based on Chinese Supercomputer)

Data availability The data used to support the findings of this study are available from the corresponding author upon request.

Declarations

Conflict of interest We declare that the authors have no competing interests as defined by Springer, or other interests that might be perceived to influence the results and/or discussion reported in this paper.

Ethical approval In this paper, the declaration is not applicable.

References

1. Abraham FF (1986) Computational statistical mechanics methodology, applications and supercomputing. Adv Phys 35(1):1–111
2. Guo Z, Saunders N, Schillé J, Miodownik A (2009) Material properties for process simulation. Mater Sci Eng A 499(1–2):7–13
3. Shen JZCWH (2022) Mechanical properties of floating bollard groove during pouring. Port Waterw Eng 07:211–216+234
4. Cui Y, Olsen KB, Jordan TH, Lee K, Zhou J, Small P, Roten D, Ely G, Panda DK, Chourasia A et al (2010) Scalable earthquake simulation on petascale supercomputers. In: SC'10: Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, pp 1–20
5. Margetts L. ParaFEM: towards a massively parallel alternative to Abaqus/Ansys for implicit solid mechanics
6. Margetts L, Evans L, Arregui D, Lever L (2014) General purpose finite element analysis of problems with billions of degrees of freedom. In: NAFEMS UK Regional Conference
7. Preis T, Virnau P, Paul W, Schneider JJ (2009) GPU accelerated Monte Carlo simulation of the 2D and 3D Ising model. J Comput Phys 228(12):4468–4477
8. Stone JE, Hardy DJ, Ufimtsev IS, Schulten K (2010) GPU-accelerated molecular modeling coming of age. J Mol Graph Model 29(2):116–125
9. Han S, Jang K, Park K, Moon S (2010) Packetshader: a GPU-accelerated software router. ACM SIGCOMM Comput Commun Rev 40(4):195–206
10. Sukhwani B, Min H, Thoennes M, Dube P, Iyer B, Brezzo B, Dillenberger D, Asaad S (2012) Database analytics acceleration using FPGAs. In: 2012 21st International Conference on Parallel Architectures and Compilation Techniques (PACT). IEEE, pp 411–420
11. Hoozemans J, Peltenburg J, Nonnemacher F, Hadnagy A, Al-Ars Z, Hofstee HP (2021) FPGA acceleration for big data analytics: challenges and opportunities. IEEE Circuits Syst Mag 21(2):30–47
12. Ma Y, Suda N, Cao Y, Vrudhula S, Seo J (2018) Alamo: FPGA acceleration of deep learning algorithms with a modularized rtl compiler. Integration 62:14–23
13. Fu H, Liao J, Yang J, Wang L, Song Z, Huang X, Yang C, Xue W, Liu F, Qiao F et al (2016) The Sunway TaihuLight supercomputer: system and applications. Sci China Inf Sci 59(7):1–16
14. Dongarra J (2016) Sunway TaihuLight supercomputer makes its appearance. Nat Sci Rev 3(3):265–266
15. Lin J, Wen M, Meng D, Liu X, Nukada A, Matsuoka S (2018) Optimizing preconditioned conjugate gradient on TaihuLight for OpenFOAM. In: 2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID). IEEE, pp 273–282
16. Ao Y, Yang C, Liu F, Yin W, Jiang L, Sun Q (2018) Performance optimization of the HPCG benchmark on the Sunway TaihuLight supercomputer. ACM Trans Archit Code Optim 15(1):1. https://doi.org/10.1145/3182177
17. Yang C, Xue W, Fu H, You H, Wang X, Ao Y, Liu F, Gan L, Xu P, Wang L, Yang G, Zheng W (2016) 10m-core scalable fully-implicit solver for nonhydrostatic atmospheric dynamics. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. SC '16

18. Zhaoxiang CPFJQ (2022) Experimental and numerical study of the tensile behavior of high-strength steel T-stub. Prog Steel Build Struct 24(05):40–50+112. https://doi.org/10.13969/j.cnki.cn31-1893.2022.05.005
19. Yu Xuan SH (2021) Review of research on hole edge crack monitoring technology of aviation structural parts. Aeronaut Sci Technol 32(12):1–17. https://doi.org/10.19452/j.issn1007-5453.2021.12.001
20. Dai Z, Wang Y, Wang F, Ming L, Zhang J et al (2022) Performance optimization and analysis of the unstructured discontinuous Galerkin solver on multi-core and many-core architectures. arXiv:2209.01877
21. Yang J, Xu Y, Yang L (2022) Taichi-LBM3D: a single-phase and multiphase lattice Boltzmann solver on cross-platform multicore CPU/GPUs. Fluids 7(8):270
22. Horikoshi M, Gerofi B, Ishikawa Y, Nakajima K (2022) Exploring communication-computation overlap in parallel iterative solvers on manycore cpus using asynchronous progress control. In: International Conference on High Performance Computing in Asia-Pacific Region Workshops, pp 29–39
23. Badia JM, Amor-Martin A, Belloch JA, Garcia-Castillo LE (2022) Strategies to parallelize a finite element mesh truncation technique on multi-core and many-core architectures. J Supercomput 1–17. https://link.springer.com/article/10.1007/s11227-022-04975-6
24. Smith I, Margetts L, Beer G, Dünser C (2007) Parallelising the boundary element method using ParaFEM. In: Proceedings of the Tenth International Conference on Numerical Methods in Geomechanics, NUMOG X
25. Arregui-Mena J.D, Margetts L, Lever L, Hall G, Mummery P (2014) Stochastic thermomechanical analysis of nuclear graphite using parafem. In: UK Conference of the Association for Computational Mechanics in Engineering
26. Tsamos A.G, Margetts L, Jivkov AP (2016) Implementation of a cohesive zone model into the open source finite element software ParaFEM. In: Proceedings of the 24th UK Conference of the Association for Computational Mechanics in Engineering: 31 March–01 April 2016, Cardiff University, Cardiff
27. Pan J, Xiao L, Tian M, Liu T, Wang L (2021) Heterogeneous multi-core optimization of mumps solver and its application. In: Proceedings of the 2021 ACM International Conference on Intelligent Computing and Its Emerging Applications, pp 122–127
28. Fang J, Fu H, Zhao W, Chen B, Zheng W, Yang G (2017) swdnn: a library for accelerating deep learning applications on Sunway TaihuLight. In: 2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS). IEEE, pp 615–624
29. Dong W, Kang L, Quan Z, Li K, Li K, Hao Z, Xie X-H (2016) Implementing molecular dynamics simulation on Sunway TaihuLight system. In: 2016 IEEE 18th International Conference on High Performance Computing and Communications; IEEE 14th International Conference on Smart City; IEEE 2nd International Conference on Data Science and Systems (HPCC/SmartCity/DSS). IEEE, pp 443–450
30. Li L, Fang J, Fu H, Jiang J, Zhao W, He C, You X, Yang G (2018) swcaffe: a parallel framework for accelerating deep learning applications on Sunway TaihuLight. In: 2018 IEEE International Conference on Cluster Computing (CLUSTER). IEEE, pp 413–422
31. Duan X, Gao P, Zhang T, Zhang M, Liu W, Zhang W, Xue W, Fu H, Gan L, Chen D et al (2018) Redesigning LAMMPS for peta-scale and hundred-billion-atom simulation on Sunway TaihuLight. In: SC18: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, pp 148–159
32. Lin H, Tang X, Yu B, Zhuo Y, Chen W, Zhai J, Yin W, Zheng W (2017) Scalable graph traversal on Sunway TaihuLight with ten million cores. In: 2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS). IEEE, pp 635–645
33. Tian M, Wang J, Zhang Z, Du W, Pan J, Liu T (2022) swSuperLU: a highly scalable sparse direct solver on Sunway manycore architecture. J Supercomput 78(9):11441–11463
34. Xu K, Duan X, Müller A, Kobus R, Schmidt B, Liu W (2022) Fmapper: scalable read mapper based on succinct hash index on SunWay TaihuLight. J Parallel Distrib Comput 161:72–82
35. Ye Y, Song Z, Zhou S, Liu Y, Shu Q, Wang B, Liu W, Qiao F, Wang L (2022) swnemo_v4.0: an ocean model NEMO for the next generation Sunway supercomputer. Geosci Model Dev Discuss 2022
36. Zhu Q, Luo H, Yang C, Ding M, Yin W, Yuan X (2021) Enabling and scaling the hpcg benchmark on the newest generation Sunway supercomputer with 42 million heterogeneous cores. In:

Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp 1–13
37. Badia S, Martín AF, Principe J (2016) Multilevel balancing domain decomposition at extreme scales. SIAM J Sci Comput 38(1):22–52
38. Fabien MS, Knepley MG, Mills RT, Rivière BM (2019) Manycore parallel computing for a hybridizable discontinuous Galerkin nested multigrid method. SIAM J Sci Comput 41(2):73–96

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.